

TÀI LIỆU TỔNG HỢP

HƯỚNG DẪN LẬP TRÌNH MISRA C

MISRA C:2012 & MISRA C:2023 — Quy tắc 1 đến 125

Tổng hợp từ 4 phiên (Session 1–4), dịch sang tiếng Việt phần lý thuyết;
phần mã nguồn (code) được giữ nguyên bản gốc.

Session 1 — Quy tắc 1 đến 38

MISRA C là gì

- MISRA C là một tập hợp các quy tắc lập trình (coding guidelines) dành cho ngôn ngữ C.
- MISRA là viết tắt của Motor Industry Software Reliability Association (Hiệp hội Độ tin cậy Phần mềm ngành Công nghiệp Ô tô).
- MISRA C đưa ra các quy tắc và khuyến nghị nhằm giúp viết mã C một cách có cấu trúc, nhất quán và đáng tin cậy. Các hướng dẫn này bao quát nhiều khía cạnh của lập trình C, bao gồm cú pháp, ngữ nghĩa và các đặc tính của ngôn ngữ, với mục tiêu giảm thiểu lỗi và lỗ hổng tiềm ẩn trong phần mềm.

Các thuật ngữ quan trọng

- Mandatory (Bắt buộc tuyệt đối) – Các hướng dẫn bắt buộc phải luôn được tuân thủ.
- Required (Bắt buộc) – Các hướng dẫn bắt buộc phải được tuân thủ, trừ khi có Deviation (ngoại lệ được phê duyệt).
- Advisory (Khuyến nghị) – Các hướng dẫn khuyến nghị được xem là thực hành tốt, nhưng việc tuân thủ không mang tính bắt buộc chặt chẽ.
- Decidable (Có thể quyết định) – Nếu một quy tắc có thể được kiểm chứng một cách chắc chắn bằng công cụ phân tích mã tĩnh (ví dụ Polyspace), tức là nằm hoàn toàn trong khả năng của công cụ.
- Undecidable (Không thể quyết định) – Một quy tắc không thể quyết định, về lý thuyết, không thể được kiểm tra bởi công cụ phân tích tĩnh.
- Single Translation Unit (Đơn vị biên dịch đơn) – Một đơn vị biên dịch về cơ bản là một tệp nguồn sau khi đã được xử lý bởi bộ tiền xử lý C, nghĩa là các tệp header trong chỉ thị `#include` được đưa vào (include) theo đúng nghĩa đen.
- System (Hệ thống) – Một quy tắc ở mức hệ thống yêu cầu phải phân tích toàn bộ chương trình để đánh giá sự tuân thủ.

Lịch sử MISRA-C

- MISRA C:2012 là phiên bản thứ ba của MISRA C, được công bố năm 2012. Bộ quy tắc được viết cho C99 và C90 nhằm cung cấp cơ sở lý giải (rationale) tốt hơn cho các hướng dẫn và mô tả chính xác hơn. Bộ quy tắc gồm 143 quy tắc (rules), 17 chỉ thị (directives).

Rule 1.1

Chương trình không được vi phạm cú pháp và các ràng buộc của C chuẩn, và không được vượt quá giới hạn dịch (translation limits) của trình biên dịch

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

LỢI ÍCH

Quy tắc này ngăn ngừa các hành vi không xác định (undefined behavior) và cải thiện khả năng đọc của mã nguồn.

KHẮC PHỤC

Không được vượt quá giới hạn dịch (translation limits) của trình biên dịch.

THAM KHẢO

MISRA C:2012 Guidelines for the use of the C language in critical systems — Section 8: Rules, Rule 1.1

Rule 1.2

Không nên sử dụng các phần mở rộng ngôn ngữ (language extensions)

Phân loại: Khuyến nghị (Advisory) | Phân tích: Không thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nhiều trình biên dịch cung cấp các phần mở rộng — là những tính năng bổ sung không được định nghĩa trong chuẩn nhưng được hỗ trợ bởi trình biên dịch cụ thể đó. Ví dụ: GNU Compiler Collection (GCC) thường cung cấp các phần mở rộng vượt ra ngoài chuẩn C.

```
#include <stdio.h> int main()
{ int num1 = 10;
  int num2 = 20; // Violation: Using a language extension (C99-style inline
  declaration)

  for (int i = 0; i < 5; i++)
  {
    printf("Iteration %d: %d\n", i+1, num1 + num2);
  }
  return 0;
}
```

KHẮC PHỤC

Di chuyển khai báo biến đếm vòng lặp ra ngoài vòng lặp.

```
#include <stdio.h> int main()
{ int num1 = 10;
  int num2 = 20; // Violation: Using a language extension (C99-style inline
  declaration)

  int i; // Loop counter declaration moved outside the loop

  for (i = 0; i < 5; i++)
  {
    printf("Iteration %d: %d\n", i+1, num1 + num2); // Loop counter initialization
    moved outside the loop
  }
  return 0;
}
```

Rule 1.3

Không được xảy ra hành vi không xác định (undefined behaviour) hoặc hành vi không xác định nghiêm trọng chưa được quy định (critical unspecified behaviour)

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Hành vi này có thể xảy ra khi chương trình thực hiện các thao tác không được chuẩn quy định, chẳng hạn như chia cho 0, truy cập biến chưa được khởi tạo, hoặc tràn số nguyên có dấu.

```
int main()
{
  int p = 5;
  int q = 0;
  int Multiplication = q * p;
  int Division = p / q; // Division by zero, leading to undefined behavior ; /*
  Non-compliant */
  printf("Result: %d\n %d\n", Multiplication, Division);
  return 0;
}
```

Rule 2.1

Dự án không được chứa mã không thể truy cập tới (unreachable code)

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Đây là các đoạn mã không bao giờ có thể được thực thi trong bất kỳ hoàn cảnh nào khi chương trình chạy. Mã không thể truy cập tới thường xuất hiện do lỗi logic, luồng điều khiển sai, hoặc các câu lệnh dư thừa.

```
int main()
{
    int x = 10;
    if (x > 100)
    {
        printf("x is greater than 100\n");
    }
    else
    {
        printf("x is less than or equal to 100\n");
    }

    printf("This line is unreachable\n"); // Unreachable code
    return 0;
}
```

Rule 2.2

Không được có mã chết (dead code)

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Mã chết là các phần của chương trình được viết ra nhưng không bao giờ được thực thi trong quá trình chạy. Điều này có thể do các câu lệnh điều kiện không bao giờ đúng, lời gọi hàm không thể truy cập tới, hoặc biến được khai báo nhưng không bao giờ sử dụng.

```
int main() {
    int x = 10;

    if (x > 100)
    {
        printf("x is greater than 100\n");
    }
    else {
        printf("x is less than or equal to 100\n");
    }

    // Dead code below
    int y = 20; // Variable declared but never used
    return 0;
}
```

Rule 2.3

Dự án không nên chứa các khai báo kiểu (type declarations) không được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Đây là các khai báo kiểu dữ liệu (ví dụ struct, enum, typedef) được định nghĩa nhưng không được sử dụng ở bất kỳ đâu trong dự án. Các khai báo này có thể phát sinh do tái cấu trúc mã, thay đổi yêu cầu, hoặc sơ suất trong quá trình phát triển.

```
// Unused type declaration
typedef struct {
    int x;
```

```

    int y;
} Point;
int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;

    printf("Sum: %d\n", sum);
    return 0;
}

```

Rule 2.4

Dự án không nên chứa các khai báo tag không được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Đây là các khai báo kiểu dữ liệu (ví dụ struct, enum, typedef) được định nghĩa nhưng không được sử dụng ở bất kỳ đâu trong dự án. Các khai báo này có thể phát sinh do tái cấu trúc mã, thay đổi yêu cầu, hoặc sơ suất trong quá trình phát triển.

```

// Unused union tag declaration
union Data {
    int i;
    float f;
};
int main() {
    int a = 10;
    int b = 20;
    int sum = a + b;

    printf("Sum: %d\n", sum);
    return 0;
}

```

Rule 2.5

Dự án không nên chứa các khai báo macro không được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Đây là các định nghĩa macro được khai báo nhưng không được sử dụng ở bất kỳ đâu trong dự án.

```

// Used macro declaration
#define SQUARE(x) ((x) * (x))

// Unused macro declaration
#define UNUSED_MACRO 100

int main() {
    int a = 5;
    int b = SQUARE(a); // Macro is used
    printf("Result: %d\n", b);

    return 0;
}

```

Rule 2.6

Một hàm không nên chứa các khai báo nhãn (label) không được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Để tuân thủ quy tắc, khai báo nhãn không sử dụng nên được loại bỏ hoặc được dùng làm đích cho luồng điều khiển trong hàm.

```
#include <stdio.h>

void exampleFunction() {
    int x = 10;

    // Unused label declaration
    unused_label:
        printf("Value of x: %d\n", x);
}

int main() {
    exampleFunction();
    return 0;
}
```

KHẮC PHỤC

Trong ví dụ đã sửa dưới đây, nhãn `used_label` được khai báo trong hàm `exampleFunction` và được sử dụng làm đích cho luồng điều khiển thông qua câu lệnh `goto`.

```
#include <stdio.h>

void exampleFunction() {
    int x = 10;
    // Used label declaration
    used_label:
        printf("Value of x: %d\n", x);
    // Usage of the label as a target for control flow
    goto used_label;
}

int main() {
    exampleFunction();
    return 0;
}
```

Rule 2.7

Không nên có tham số hàm không được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Tham số không được sử dụng có thể cho thấy sự kém hiệu quả tiềm ẩn trong mã nguồn, vì dữ liệu không cần thiết có thể được truyền đi mà không thực sự được dùng.

```
#include <stdio.h>

// Function with an unused parameter

void exampleFunction(int x, int y) {
    printf("Value of x: %d\n", x);
    // Parameter y is declared but not used
}

int main() {
    exampleFunction(10, 20);
    return 0;
}
```

Rule 3.1

Chuỗi ký tự `/*` và `//` không được sử dụng bên trong một chú thích (comment)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nhằm ngăn ngừa sự nhầm lẫn hoặc lỗi tiềm ẩn do chú thích lồng nhau hoặc việc kết thúc chú thích ngoài ý muốn.

```
x = y // /*
+ z
// */
;
```

Đoạn mã trên cho kết quả $x = y + z$; nhưng lẽ ra sẽ chỉ là $x = y$;

Rule 3.2

Không được sử dụng nối dòng (line-splicing) trong chú thích //

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C99

Nối dòng (\\) được sử dụng bên trong một chú thích //, điều này không được phép.

```
// This is an invalid comment with line-splicing \
int x = 5 + // Line-splicing used within // comment \
          10; // This line continues the comment

// This is a valid comment without line-splicing
int x = 5 + // Line-splicing is not used within // comments
          10;
```

Rule 4.1

Các chuỗi thoát (escape sequence) bất phân và thập lục phân phải được kết thúc đúng cách

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Các chuỗi thoát bất phân và thập lục phân bên trong chuỗi ký tự cần được kết thúc đúng cách.

```
// Non-compliant: Improperly terminated hexadecimal escape sequence
const char *s1 = "\x41g";
// Compliant: Hexadecimal escape sequence terminated by end of literal
const char *s2 = "\x41" "g";
// Compliant: Hexadecimal escape sequences terminated by another escape
const char *s3 = "\x41\x67";
// Non-compliant: Improperly terminated octal escape sequence
int c1 = '\141t';
// Compliant: Octal escape sequence terminated by another escape
int c2 = '\141\t';
```

Rule 4.2

Không nên sử dụng trigraph

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Trigraph là các chuỗi ba ký tự đại diện cho một ký tự duy nhất. Chúng được đưa vào ngôn ngữ C nhằm cho phép nhập các ký tự đặc biệt trên những bàn phím thiếu một số ký hiệu nhất định.

```
int main() {
    printf("This is a backslash: ??/"); // Using trigraph instead of backslash
    return 0;
}
```

Rule 5.1

Các định danh bên ngoài (external identifiers) phải khác biệt nhau

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc 5.1 quy định rằng các định danh bên ngoài, như tên biến, tên hàm và tag cấu trúc, phải khác biệt nhau.

```
#include <stdio.h>
// Violation: Non-distinct external identifiers
int variable = 10;
int variable = 20; // Attempting to declare another variable with the same name
void function() {
    printf("Hello, world!\n");
}
void function() { // Attempting to declare another function with the same name
    printf("Hello again!\n");
}
int main() {
    variable = 30; // Using the non-distinct variable identifier
    function(); // Calling the non-distinct function identifier
    return 0;
}
```

Rule 5.2

Các định danh khai báo trong cùng phạm vi (scope) và không gian tên (namespace) phải khác biệt nhau

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc quy định rằng các định danh khai báo trong cùng phạm vi và không gian tên phải khác biệt nhau.

```
void exampleFunction() {
    int localVar = 10;
    int localVar = 20; // Violation: Attempting to declare another identifier
    with the same name
    printf("Local variable: %d\n", localVar);
}

int main() {
    int localVar = 30; // Violation: Identifier already declared in the same
    namespace
    printf("Local variable: %d\n", localVar);
    return 0;
}
```

Rule 5.3

Một định danh khai báo trong phạm vi bên trong không được che khuất một định danh khai báo trong phạm vi bên ngoài

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc 5.3 quy định rằng một định danh khai báo trong phạm vi bên trong không được che khuất một định danh khai báo trong phạm vi bên ngoài.

```
void fn1 ( void )
{
    int16_t i; /* Declare an object "i" */
    {
        int16_t i; /* Non-compliant - hides previous "i" */
        i = 3; /* Could be confusing as to which "i" this refers */
    }
}
```

```
}  
}
```

Rule 5.4

Các định danh macro phải khác biệt nhau

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Mỗi định danh macro phải là duy nhất và khác biệt trong cùng không gian tên. Nếu cần, hãy chọn tên khác nhau cho các macro để tránh xung đột và đảm bảo tính rõ ràng của mã.

```
#include <stdio.h>  
  
#define MAX_SIZE 100  
#define MAX_SIZE 200 // Violation: Attempting to redefine the macro identifier  
MAX_SIZE  
  
int main() {  
    int array[MAX_SIZE]; // Using the MAX_SIZE macro  
    return 0;  
}
```

Rule 5.5

Các định danh phải khác biệt với tên macro

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Quy tắc 5.5 quy định rằng các định danh phải khác biệt với tên macro.

```
#include <stdio.h>  
  
#define MAX_SIZE 100  
  
int main() {  
    int MAX_SIZE = 200; // Violation: Identifier has the same name as a macro  
    printf("Array size: %d\n", MAX_SIZE); // Accessing the identifier MAX_SIZE  
    return 0;  
}
```

Rule 5.6

Tên typedef phải là một định danh duy nhất

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc 5.6 quy định rằng tên typedef phải là một định danh duy nhất.

```
#include <stdio.h>  
// Define a typedef name for a custom type  
typedef struct {  
    int x;  
    int y;  
} Point;  
// Violation: Reusing the same typedef name for a different type  
typedef struct {  
    float x;  
    float y;  
} Point;  
int main() {  
    // Declaration of variables is omitted for brevity  
    return 0;  
}
```

Rule 5.7

Tên tag phải là một định danh duy nhất

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc 5.7 quy định rằng tên tag phải là một định danh duy nhất.

```
#include <stdio.h>
// Define a structure type with the tag name 'Point'
struct Point {
    int x;
    int y;
};
// Violation: Reusing the same tag name 'Point' for a different structure type
struct Point {
    float x;
    float y;
};
int main() {
    // Declaration of variables is omitted for brevity
    return 0;
}
```

Rule 5.8

Các định danh định nghĩa đối tượng hoặc hàm có liên kết bên ngoài (external linkage) phải là duy nhất

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc 5.8 quy định rằng các định danh định nghĩa đối tượng hoặc hàm có liên kết bên ngoài phải là duy nhất.

```
// File: module1.c
int globalVariable; // Identifier defining an object with external linkage

void function() {} // Identifier defining a function with external linkage

// File: module2.c
int globalVariable; // Violation: Reusing the same identifier for a different
object with external linkage

void function() {} // Violation: Reusing the same identifier for a different
function with external linkage
```

Rule 5.9

Các định danh định nghĩa đối tượng hoặc hàm có liên kết nội bộ (internal linkage) nên là duy nhất

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Quy tắc 5.9 quy định rằng các định danh định nghĩa đối tượng hoặc hàm có liên kết nội bộ nên là duy nhất.

```
// File: module.c
static int moduleVariable; // Identifier defining an object with internal linkage

static void moduleFunction() {} // Identifier defining a function with internal
linkage

// Violation: Reusing the same identifier for a different object with internal
linkage
static int moduleVariable;
```

```
// Violation: Reusing the same identifier for a different function with internal linkage
static void moduleFunction() {}
```

Rule 6.1

Bit-field chỉ được khai báo với kiểu phù hợp

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Bit-field là các thành viên của struct hoặc union chỉ định một số lượng bit nhất định được cấp phát cho một trường cụ thể.

```
#include <stdio.h>

// Violation: Bit-field declared with an inappropriate type (float)
struct Flags {
    float flag : 1;
};

int main() {
    // Declaration of variables is omitted for brevity
    return 0;
}
```

Rule 6.2

Bit-field có tên với một bit duy nhất không được là kiểu có dấu (signed)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nhằm đảm bảo rằng các bit-field có tên với một bit duy nhất không được khai báo với kiểu có dấu.

```
#include <stdio.h>

// Define a structure with single-bit named bit-fields
struct Flags {
    unsigned int flag1 : 1; // Single-bit named bit-field of unsigned type
    unsigned int flag2 : 1; // Single-bit named bit-field of unsigned type
};

int main() {
    // Declaration of variables is omitted for brevity
    return 0;
}
```

Rule 7.1

Không được sử dụng hằng số bát phân (octal constants)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Lập trình viên khi viết các hằng số có số 0 ở đầu có thể kỳ vọng chúng được hiểu là hằng số thập phân.

```
#include <stdint.h>

extern uint16_t code[10];

int main() {
    code[1] = 109; // Compliant - decimal 109
    code[2] = 100; // Compliant - decimal 100
    code[3] = 052; // Compliant - decimal 42 (instead of octal 052)
    code[4] = 071; // Compliant - decimal 57 (instead of octal 071)
}
```

```
    return 0;
}
```

Rule 7.2

Hậu tố "u" hoặc "U" phải được áp dụng cho tất cả các hằng số nguyên biểu diễn ở kiểu không dấu (unsigned)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Để tránh nhầm lẫn và làm cho mã rõ ràng hơn, đặc biệt khi làm việc với kiểu không dấu, nên thêm hậu tố 'u' hoặc 'U' vào các hằng số nguyên có chủ đích là không dấu.

Constant	Type	Compliance
32767	signed int	Compliant
32768	signed long	Compliant
32768u	unsigned int	Compliant
0x8000U	unsigned int	Compliant (Added 'U' suffix)
0x8000u	unsigned int	Compliant

Rule 7.3

Không được sử dụng chữ cái thường "l" làm hậu tố literal

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Sử dụng hậu tố in hoa "L" giúp loại bỏ sự nhập nhằng tiềm ẩn giữa "1" (chữ số 1) và "l" (chữ cái "el") khi khai báo literal.

```
const int64_t a = 0L;
const int64_t b = 0l; /* Non-compliant */
const uint64_t c = 0Lu;
const uint64_t d = 0lU; /* Non-compliant */
const uint64_t e = 0ULL;
const uint64_t f = 0Ull; /* Non-compliant */
const int128_t g = 0LL;
const int128_t h = 0ll; /* Non-compliant */
const float128_t m = 1.2L;
const float128_t n = 2.4l; /* Non-compliant */
```

Rule 7.4

Không được gán một chuỗi ký tự literal cho một đối tượng trừ khi kiểu của đối tượng là "con trỏ tới char có const-qualified"

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Mọi nỗ lực chỉnh sửa một chuỗi ký tự literal đều dẫn đến hành vi không xác định.

```
char *name1 ( void )
{
    return ( "MISRA" ); /* Non-compliant - return type is not
    * const-qualified */
}
const char *name2 ( void )
{
    return ( "MISRA" ); /* Compliant */
}
```

Rule 8.1

Các kiểu phải được chỉ định một cách tường minh (explicitly)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90

Chuẩn C90 cho phép bỏ qua kiểu trong một số trường hợp, khi đó kiểu `int` sẽ được ngầm định (implicitly) chỉ định.

```
extern x; /* Non-compliant - implicit int type */
extern int16_t x; /* Compliant - explicit type */
const y; /* Non-compliant - implicit int type */
const int16_t y; /* Compliant - explicit type */
```

Các ví dụ sau minh họa khai báo kiểu hàm tuân thủ và không tuân thủ:

```
extern f ( void ); /* Non-compliant - implicit
* int return type */
extern int16_t f ( void ); /* Compliant */
extern void g ( char c, const k ); /* Non-compliant - implicit
* int for parameter k */
extern void g ( char c, const int16_t k ); /* Compliant */
```

Rule 8.2

Kiểu hàm phải ở dạng prototype với tham số có tên

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Một function prototype cung cấp thông tin về giao diện của hàm, gồm kiểu trả về, tên và kiểu các tham số. Prototype với tham số có tên chỉ rõ kiểu và tên tham số, giúp khai báo hàm rõ ràng và tự giải thích hơn.

```
/* Compliant */
extern int16_t func1 ( int16_t n );
/* Non-compliant - parameter name not specified */
extern void func2 ( int16_t );
/* Non-compliant - not in prototype form */
static int16_t func3 ( );
/* Compliant - prototype specifies 0 parameters */
static int16_t func4 ( void );
/* Compliant */
int16_t func1 ( int16_t n )
{
    return n;
}
```

Rule 8.3

Tất cả các khai báo của một đối tượng hoặc hàm phải sử dụng cùng tên và cùng bộ định tính kiểu (type qualifiers)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Vi phạm Quy tắc 8.3 xảy ra khi có sự không nhất quán về tên hoặc bộ định tính kiểu giữa các khai báo khác nhau của cùng một đối tượng hoặc hàm.

```
extern int16_t func ( int16_t num, int16_t den );
/* Non-compliant - parameter names do not match */
int16_t func ( int16_t den, int16_t num )
{
    return num / den;
}
```

Rule 8.4

Một khai báo tương thích phải hiện diện khi một đối tượng hoặc hàm có liên kết bên ngoài được định nghĩa

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu một khai báo cho đối tượng hoặc hàm hiện diện tại nơi đối tượng/hàm đó được định nghĩa, trình biên dịch phải kiểm tra rằng khai báo và định nghĩa là tương thích.

```
extern int16_t count;
int16_t count = 0; /* Compliant */
extern uint16_t speed = 6000u; /* Non-compliant - no declaration
* prior to this definition */
uint8_t pressure = 101u; /* Non-compliant - no declaration
prior to this definition */
```

Phiên bản tuân thủ:

```
extern int16_t count;
int16_t count = 0; // Compliant
extern uint16_t speed; // Declaration added
uint16_t speed = 6000u; // Compliant
extern uint8_t pressure; // Declaration added
uint8_t pressure = 101u; // Compliant
```

Rule 8.5

Một đối tượng hoặc hàm bên ngoài chỉ được khai báo đúng một lần, trong đúng một tệp

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Thông thường, khai báo duy nhất sẽ được đặt trong một tệp header, tệp này được include trong bất kỳ đơn vị biên dịch nào cần định nghĩa hoặc sử dụng định danh đó.

```
/* featureX.h */
extern int16_t a; /* Declare a */
/* file.c */
#include "featureX.h"
int16_t a = 0; /* Define a */
```

Rule 8.6

Một định danh có liên kết bên ngoài phải có đúng một định nghĩa bên ngoài

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Nhiều định nghĩa trong các tệp khác nhau là không được phép theo quy tắc này, kể cả khi các định nghĩa đó giống hệt nhau.

```
/* file1.c */
int16_t i = 10;
/* file2.c */
int16_t i = 20; /* Non-compliant - two definitions of i */
```

Trong ví dụ dưới, đối tượng j có một định nghĩa tạm thời (tentative) và một định nghĩa bên ngoài — tuân thủ:

```
/* file3.c */
int16_t j; /* Tentative definition */
int16_t j = 1; /* Compliant - external definition */
```

Ví dụ sau đây không tuân thủ vì đối tượng k có hai định nghĩa bên ngoài. Định nghĩa tạm thời trong file4.c trở thành định nghĩa bên ngoài vào cuối đơn vị biên dịch.

```
/* file4.c */
int16_t k; /* Tentative definition — Becomes external */
/* file5.c */
int16_t k = 0; /* External definition */
```

Rule 8.7

Hàm và đối tượng không nên được định nghĩa với liên kết bên ngoài nếu chúng chỉ được tham chiếu trong một đơn vị biên dịch

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Hệ thống | Áp dụng: C90, C99

Liên kết bên ngoài (external linkage) nghĩa là hàm hoặc đối tượng có thể được truy cập từ các đơn vị biên dịch khác. Ngược lại, liên kết nội bộ (internal linkage) nghĩa là hàm/đối tượng chỉ có thể truy cập trong cùng đơn vị biên dịch.

```
// File: math_utils.c
#include "math_utils.h"

int add(int a, int b) {
    return a + b;
}

// File: math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

int add(int a, int b); // Declaration of the function

#endif // MATH_UTILS_H
```

Ví dụ (tiếp) — tệp gọi hàm:

```
// File: main.c
#include "math_utils.h"
#include <stdio.h>
extern int add(int a, int b); // Declaration with external linkage
int main() {
    int result = add(3, 4);
    printf("Result: %d\n", result);
    return 0;
}
```

Rule 8.8

Bộ định danh lớp lưu trữ static phải được sử dụng trong mọi khai báo của các đối tượng và hàm có liên kết nội bộ

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Yêu cầu bắt buộc sử dụng bộ định danh lớp lưu trữ static trong mọi khai báo của các đối tượng và hàm có liên kết nội bộ.

```
static int32_t x = 0; /* definition: internal linkage */
extern int32_t x; /* Non-compliant */
static int32_t f ( void ); /* declaration: internal linkage */
int32_t f ( void ) /* Non-compliant */
{
    return 1;
}
static int32_t g ( void ); /* declaration: internal linkage */
extern int32_t g ( void ) /* Non-compliant */
{
    return 1;
}
```

Session 2 — Quy tắc 39 đến 53

Rule 8.9

Một đối tượng nên được định nghĩa ở phạm vi khối (block scope) nếu định danh của nó chỉ xuất hiện trong một hàm duy nhất

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Hệ thống

Định nghĩa một đối tượng ở phạm vi khối làm giảm khả năng đối tượng đó bị truy cập ngoài ý muốn, đồng thời thể hiện rõ ý định rằng nó không nên được truy cập ở nơi khác.

```
void func ( void )
{
    int32_t i;
    for ( i = 0; i < N; ++i )
    {
    }
}
```

Trong ví dụ tuân thủ này, `i` được khai báo ở phạm vi khối vì nó là biến đếm vòng lặp. Không có nhu cầu để các hàm khác trong cùng tệp sử dụng cùng đối tượng đó cho mục đích khác.

Rule 8.10

Một hàm inline phải được khai báo với lớp lưu trữ static

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Quy tắc 8.10 quy định rằng một hàm inline nên được khai báo với lớp lưu trữ static.

Hàm inline là những hàm có phần định nghĩa nhỏ gọn và được thay thế trực tiếp tại vị trí gọi hàm.

Lưu ý: một hàm inline có thể được cung cấp cho nhiều đơn vị biên dịch bằng cách đặt định nghĩa của nó trong một tệp header.

```
// File: math_utils.h
#ifndef MATH_UTILS_H
#define MATH_UTILS_H

static inline int add(int a, int b) {
    return a + b;
}

#endif // MATH_UTILS_H
```

Rule 8.11

Khi một mảng có liên kết bên ngoài được khai báo, kích thước của nó nên được chỉ định tường minh

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Mặc dù có thể khai báo một mảng với kiểu chưa hoàn chỉnh (incomplete type) và truy cập các phần tử của nó, nhưng sẽ an toàn hơn nếu kích thước của mảng được xác định tường minh.

```
extern int32_t array1[ 10 ]; /* Compliant */
extern int32_t array2[ ]; /* Non-compliant */
```

Rule 8.12

Trong một danh sách enumerator, giá trị của một hằng số liệt kê được chỉ định ngầm định phải là duy nhất

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Một hằng số liệt kê được chỉ định tường minh sẽ có giá trị của biểu thức hằng liên kết với nó.

```
/* Non-compliant */
enum Color {
    RED = 0,      // Value: 0
    GREEN = 1,    // Value: 1
    BLUE = 1,     // Value: 2 (Violation: Not unique)
    YELLOW = 3    // Value: 3
};
/* Compliant */
enum Color {
    RED = 0,      // Value: 0
    GREEN = 1,    // Value: 1
    BLUE = 2,     // Value: 2 (Distinct from GREEN)
    YELLOW = 3    // Value: 3
};
```

Rule 8.13

Một con trỏ nên trỏ tới kiểu *const-qualified* bất cứ khi nào có thể

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Hệ thống

Quy tắc này khuyến khích thực hành tốt nhất bằng cách đảm bảo con trỏ luôn sử dụng kiểu *const-qualified* khi có thể.

Trong ví dụ không tuân thủ dưới đây, `p` không được dùng để chỉnh sửa đối tượng nhưng kiểu mà nó trỏ tới lại không phải *const-qualified*.

Sử dụng con trỏ *const-qualified* giúp cải thiện tính an toàn, khả năng bảo trì và khả năng đọc của mã, bằng cách đảm bảo đúng hành vi dự kiến và ngăn chặn việc vô tình chỉnh sửa dữ liệu.

```
uint16_t f ( uint16_t *p )
{
    return *p;
}
/* Đoạn mã sẽ tuân thủ nếu hàm được định nghĩa như sau: */
uint16_t f ( const uint16_t *p )
```

Ví dụ khác:

```
#include <stdio.h>

int main() {
    const int number = 10; // Declare a const-qualified variable
    const int *ptr = &number; // Declare a const-qualified pointer

    // Attempting to modify the data pointed to by the pointer will result in a
    compilation error
    // *ptr = 20; // This line will result in a compilation error

    printf("Value of number: %d\n", *ptr); // Output: Value of number: 10

    return 0;
}
```

Rule 8.14

Không được sử dụng bộ định tính kiểu *restrict*

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Có rủi ro đáng kể rằng trình biên dịch sẽ sinh mã không hoạt động như mong đợi nếu restrict được sử dụng không đúng cách.

Ví dụ dưới đây không tuân thủ vì một hàm được định nghĩa có sử dụng restrict.

```
void add_arrays(int n, int *restrict a, int *restrict b, int *restrict result) {
    for (int i = 0; i < n; i++) {
        result[i] = a[i] + b[i]; // Aliasing occurs when two or more pointers
        // refer to the same memory location
        a[i] = result[i]; // Potential aliasing issue
    }
}
```

Điều này dẫn đến hành vi không xác định, vì việc chỉnh sửa 'a' cũng sẽ ảnh hưởng đến result, gây ra hành vi không mong muốn hoặc crash.

Khi sử dụng restrict, về bản chất bạn đang nói với trình biên dịch rằng con trỏ này bị "giới hạn" (restricted) và không aliasing (trùng vùng nhớ) với bất kỳ con trỏ nào khác trong cùng phạm vi.

Rule 9.1

Giá trị của một đối tượng có thời gian sống lưu trữ tự động (automatic storage duration) không được đọc trước khi nó được gán giá trị

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Không thể quyết định, Hệ thống

Theo chuẩn, các đối tượng có thời gian sống lưu trữ tĩnh (static storage duration) sẽ tự động được khởi tạo về 0 trừ khi được khởi tạo tường minh. Các đối tượng có thời gian sống lưu trữ tự động thì không được tự động khởi tạo và do đó có thể mang giá trị không xác định.

```
void f ( bool_t b, uint16_t *p )
{
    if ( b )
    {
        *p = 3U;
    }
}

void g ( void )
{
    uint16_t u;          // uint16_t u = 0U; // Initialize u to a known value
    f ( false, &u );
    if ( u == 3U )
    {
        /* Non-compliant - u has not been assigned a value */
    }
}
```

Rule 9.2

Bộ khởi tạo cho một aggregate hoặc union phải được đặt trong dấu ngoặc nhọn

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Việc dùng dấu ngoặc nhọn để chỉ rõ khởi tạo của các đối tượng con giúp mã rõ ràng hơn, và buộc lập trình viên phải cân nhắc việc khởi tạo các phần tử trong các cấu trúc dữ liệu phức tạp như mảng đa chiều hoặc mảng cấu trúc.

```
int16_t y[ 3 ][ 2 ] = { 1, 2, 0, 0, 5, 6 }; /* Non-compliant */
int16_t y[ 3 ][ 2 ] = { { 1, 2 }, { 0 }, { 5, 6 } }; /* Compliant */
int16_t y[ 3 ][ 2 ] = { { 1, 2 }, { 0, 0 }, { 5, 6 } }; /* Compliant */
struct s1 {
    uint16_t len;
    char buf[ 8 ]; /* Compliant - buf initialized implicitly */
} s[ 3 ] = {
```

```
{ 5u, { 'a', 'b', 'c', 'd', 'e', '\\0', '\\0', '\\0' } },
{ 2u, { 0 } },
{ .len = 0u }
}; /* Compliant - s[ ] fully initialized */
```

Rule 9.3

Mảng không được khởi tạo một phần (partially initialized)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Việc khởi tạo tường minh cho từng phần tử của mảng giúp thể hiện rõ rằng mọi phần tử đã được xem xét.

```
/* Compliant */
int32_t x[ 3 ] = { 0, 1, 2 };

/* Non-compliant - y[ 2 ] is implicitly initialized */
int32_t y[ 3 ] = { 0, 1 };

/* Non-compliant - t[ 0 ] and t[ 3 ] are implicitly initialized */
float32_t t[ 4 ] = { [ 1 ] = 1.0f, 2.0f };

/* Compliant - designated initializers for sparse matrix */
float32_t z[ 50 ] = { [ 1 ] = 1.0f, [ 25 ] = 2.0f };
```

Rule 9.4

Một phần tử của đối tượng không được khởi tạo nhiều hơn một lần

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Để cho phép mảng và cấu trúc thưa (sparse), việc chỉ khởi tạo những phần tử cần thiết cho ứng dụng là chấp nhận được.

```
/* Các phần tử khởi tạo được chỉ định (designated initializer) lặp lại sẽ ghi đè
các giá trị trước đó
* Non-compliant - a3 is -5, -4, -2, 0, -1
*/
int16_t a3[ 5 ] = { [ 0 ] = -5, [ 1 ] = -4, [ 2 ] = -3,
[ 2 ] = -2, [ 4 ] = -1 };

/* Hành vi tương tự khi dùng designated initializer
* Compliant - a2 is -5, -4, -3, -2, -1
*/
int16_t a2[ 5 ] = { [ 0 ] = -5, [ 1 ] = -4, [ 2 ] = -3,
[ 3 ] = -2, [ 4 ] = -1 };
```

Rule 9.5

Khi sử dụng designated initializer để khởi tạo một đối tượng mảng, kích thước của mảng phải được chỉ định tường minh

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C99

Để thể hiện rõ ý định, kích thước mảng phải được khai báo tường minh.

```
/* Non-compliant - probably unintentional to have a single element */
int a1[ ] = { [ 0 ] = 1 };
/* Compliant */
int a2[ 10 ] = { [ 0 ] = 1 };
```

Rule 10.1

Toán hạng không được có kiểu cơ bản (essential type) không phù hợp

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Việc sử dụng biểu thức có kiểu cơ bản là floating cho các toán hạng này là vi phạm ràng buộc.

Một biểu thức có kiểu cơ bản là Boolean nên luôn được sử dụng ở những nơi toán hạng được diễn giải như một giá trị Boolean.

Một toán hạng có kiểu cơ bản là Boolean không nên được sử dụng ở nơi toán hạng được diễn giải như một giá trị số.

```
f32a & 2U /* Rationale 1 - constraint violation */
f32a << 2 /* Rationale 1 - constraint violation */

cha && bla /* Rationale 2 - char type used as a Boolean value */
s8a && bla /* Rationale 2 - signed type used as a Boolean value */
f32a && bla /* Rationale 2 - floating type used as a Boolean value */

bla * blb /* Rationale 3 - Boolean used as a numeric value */
bla > blb /* Rationale 3 - Boolean used as a numeric value */
```

Một toán hạng có kiểu cơ bản là character không nên được sử dụng ở nơi toán hạng được diễn giải như một giá trị số, vì giá trị số của dữ liệu ký tự phụ thuộc vào cách triển khai (implementation-defined).

Một toán hạng có kiểu cơ bản là enum không nên được dùng trong phép toán số học vì kiểu enum sử dụng kiểu số nguyên phụ thuộc vào cách triển khai, có thể dẫn tới kết quả với kiểu không như mong đợi.

```
cha & chb /* Rationale 4 - char type used as a numeric value */
cha << 1 /* Rationale 4 - char type used as a numeric value */

ena-- /* Rationale 5 - enum type used in arithmetic operation */
ena * a1 /* Rationale 5 - enum type used in arithmetic operation */
```

Phép dịch bit (shift) và phép toán bit chỉ nên được thực hiện trên các toán hạng có kiểu cơ bản là unsigned; kết quả khi dùng trên kiểu signed phụ thuộc cách triển khai.

Toán hạng bên phải của phép dịch bit nên có kiểu cơ bản là unsigned để đảm bảo không xảy ra hành vi không xác định do dịch bit với số âm.

Một toán hạng có kiểu cơ bản là unsigned không nên được dùng làm toán hạng cho toán tử trừ một ngôi (unary minus), vì dấu của kết quả phụ thuộc vào kích thước int của cách triển khai.

```
s8a & 2 /* Rationale 6 - bitwise operation on signed type */
50 << 3U /* Rationale 6 - shift operation on signed type */

u8a << s8a /* Rationale 7 - shift magnitude uses signed type */
u8a << -1 /* Rationale 7 - shift magnitude uses signed type */

-u8a /* Rationale 8 - unary minus on unsigned type */
```

Rule 10.2

Các biểu thức có kiểu cơ bản là character không được sử dụng không phù hợp trong phép cộng và phép trừ

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Hướng dẫn này nhằm ngăn ngừa sự nhầm lẫn tiềm ẩn và hành vi không mong muốn khi trộn ký tự với các phép toán số, vì giá trị số của dữ liệu ký tự phụ thuộc vào cách triển khai.

```
s16a - 'a'
'0' + f32a
cha + ':'
cha - ena
```

Rule 10.3

Giá trị của một biểu thức không được gán cho một đối tượng có kiểu cơ bản hẹp hơn hoặc khác nhóm kiểu cơ bản

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Cần đảm bảo phép gán được thực hiện giữa các kiểu tương thích, hoặc nếu cần thu hẹp hay chuyển đổi khác kiểu, việc chuyển đổi đó nên được thực hiện tường minh bằng toán tử hoặc hàm chuyển đổi kiểu phù hợp.

```
uint8_t u8a = 1.0f; /* unsigned and floating */
bool_t bla = 0; /* boolean and signed */
cha = 7; /* character and signed */
u8a = 'a'; /* unsigned and character */
u8b = 1 - 2; /* unsigned and signed */
u8c += 'a'; /* u8c = u8c + 'a' assigns character to unsigned */
use_uint32 ( s32a ); /* signed and unsigned */
```

Rule 10.4

Cả hai toán hạng của một toán tử thực hiện chuyển đổi số học thông thường phải có cùng nhóm kiểu cơ bản

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn

Cần đảm bảo cả hai toán hạng của một toán tử trong biểu thức số học có cùng nhóm kiểu cơ bản. Nếu cần, hãy chuyển đổi tường minh một hoặc cả hai toán hạng về cùng một kiểu trước khi thực hiện phép toán để đảm bảo tính nhất quán và khả năng dự đoán của mã.

```
u8b + 2 /* unsigned and signed */
enb > A1 /* enum<enumb> and enum<enuma> */
ena == enb /* enum<enuma> and enum<enumb> */
u8a += cha /* unsigned and char */
```

Các ví dụ sau tuân thủ vì có cùng nhóm kiểu cơ bản:

```
enum enuma { A1, A2, A3 } ena;
enum enumb { B1, B2, B3 } enb;
ena > A1
u8a + u16b
```

Session 3 — Quy tắc 54 đến 95

Nội dung phiên 3

- Mô hình kiểu cơ bản (essential type model)
- Chuyển đổi kiểu con trỏ (pointer type conversions)
- Toán tử (operator)
- Câu lệnh điều khiển (control statement)
- Luồng điều khiển (control flow)
- Câu lệnh switch

Rule 10.5

Giá trị của một biểu thức không nên được ép kiểu (cast) sang một kiểu cơ bản không phù hợp

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Các phép ép kiểu nên tránh được liệt kê theo nhóm kiểu cơ bản mà giá trị được ép (chuyển đổi tường minh) sang.

```
(bool_t) false /* Compliant - C99 'false' is essentially Boolean */
(int32_t) 3U /* Compliant */
(bool_t) 0 /* Compliant - by exception */
(int32_t) ena /* Compliant */
(char) enc /* Compliant */
(bool_t) 3U /* Non-compliant */
```

Rule 10.6

Giá trị của một biểu thức hợp thành (composite expression) không được gán cho một đối tượng có kiểu cơ bản rộng hơn

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Gán kết quả của một biểu thức cho một biến có kiểu rộng hơn có thể dẫn tới việc mở rộng dữ liệu ngoài ý muốn và mất thông tin tiềm ẩn.

```
u16c = u16a + u16b; /* Same essential type */
u32a = (uint32_t) u16a + u16b; /* Cast causes addition in uint32_t */
/* Không tuân thủ: */
u32a = u16a + u16b; /* Implicit conversion on assignment */
```

Rule 10.7

Nếu một biểu thức hợp thành được dùng làm một toán hạng của toán tử thực hiện chuyển đổi số học thông thường, toán hạng còn lại không được có kiểu cơ bản rộng hơn

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Hạn chế chuyển đổi ngầm định trên biểu thức hợp thành nghĩa là chuỗi các phép toán số học trong một biểu thức phải được thực hiện trên cùng một kiểu cơ bản. Điều này giúp giảm khả năng gây nhầm lẫn cho lập trình viên.

```
/* Tuân thủ: */
u32a * u16a + u16b /* No composite conversion */
(u32a * u16a) + u16b /* No composite conversion */
/* Không tuân thủ: */
u32a * (u16a + u16b) /* Implicit conversion of (u16a + u16b) */
```

$u32a * u16a + u16b$ — tuân thủ vì không có chuyển đổi hợp thành: phép nhân $u32a * u16a$ được tính trước, sau đó cộng với $u16b$. Phép cộng không liên quan tới kiểu rộng hơn kết quả phép nhân.

$u32a * (u16a + u16b)$ — không tuân thủ vì $u16a + u16b$ được tính trước, cho ra kết quả kiểu `uint16_t`. Khi kết quả này nhân với $u32a$, xảy ra chuyển đổi ngầm định, nâng kết quả `uint16_t` lên `uint32_t`.

Rule 10.8

Giá trị của một biểu thức hợp thành không được ép kiểu sang một nhóm kiểu cơ bản khác hoặc kiểu cơ bản rộng hơn

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Ép kiểu sang một kiểu hẹp hơn nhưng cùng nhóm kiểu cơ bản là chấp nhận được, vì việc cắt bớt tường minh (explicit truncation) của kết quả luôn dẫn đến cùng một sự mất thông tin.

```
/* Compliant: */
(uint16_t) (u32a + u32b); // Cast after the operation on compatible types.
(uint16_t) s32a; // Simple cast without composite expression.

/* Non-compliant: */
(uint16_t) (s32a + s32b); // Different essential type categories.
(uint32_t) (u16a + u16b); // Cast to a wider essential type
```

Rule 11.1

Không được thực hiện chuyển đổi giữa một con trỏ hàm và bất kỳ kiểu nào khác

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Quy tắc MISRA C quy định không được thực hiện chuyển đổi kiểu giữa con trỏ hàm và bất kỳ kiểu nào khác, bao gồm:

Chuyển đổi con trỏ hàm thành số nguyên hoặc ngược lại.

Chuyển đổi con trỏ hàm thành con trỏ dữ liệu (ví dụ `void*`) hoặc ngược lại.

Chuyển đổi con trỏ hàm thành con trỏ của một kiểu hàm khác.

```
typedef void ( *fp16 ) ( int16_t n );
typedef void ( *fp32 ) ( int32_t n );
#include <stdlib.h> /* To obtain macro NULL */
fp16 fp1 = NULL; /* Compliant - exception 1 */
fp32 fp2 = ( fp32 ) fp1; /* Non-compliant - function
* pointer into different
* function pointer */
if ( fp2 != NULL ) /* Compliant - exception 1 */
{
}
fp16 fp3 = ( fp16 ) 0x8000; /* Non-compliant - integer into
* function pointer */
```

Rule 11.2

Không được thực hiện chuyển đổi giữa một con trỏ tới kiểu chưa hoàn chỉnh (incomplete type) và bất kỳ kiểu nào khác

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Việc chuyển đổi một con trỏ hàm sang hoặc từ: con trỏ đối tượng; con trỏ kiểu chưa hoàn chỉnh; `void*` — đều dẫn tới hành vi không xác định.

```
struct s; /* Incomplete type */
struct t; /* A different incomplete type */
```

```

struct s *sp;
struct t *tp;
int16_t *ip;
#include <stdlib.h> /* To obtain macro NULL */
ip = ( int16_t * ) sp; /* Non-compliant */
sp = ( struct s * ) 1234; /* Non-compliant */
tp = ( struct t * ) sp; /* Non-compliant - casting pointer into a
* different incomplete type */
sp = NULL; /* Compliant - exception 1 */

```

Rule 11.3

Không được thực hiện ép kiểu giữa một con trỏ tới kiểu đối tượng và con trỏ tới một kiểu đối tượng khác

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Ép kiểu một con trỏ đối tượng sang con trỏ của đối tượng khác có thể tạo ra con trỏ không được căn chỉnh (aligned) đúng, dẫn tới hành vi không xác định.

```

int32_t a = 10;
int32_t *p1 = &a;
int32_t *p2 = (int32_t *)p1; /* Compliant - casting to the same type */

int32_t a = 10;
int32_t *p1 = &a;
float *p2 = (float *)p1; /* Non-compliant - casting between different types */

```

Rule 11.4

Không nên thực hiện chuyển đổi giữa một con trỏ đối tượng và một kiểu số nguyên

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Con trỏ không nên được chuyển đổi thành số nguyên. Số nguyên không nên được chuyển đổi thành con trỏ.

```

uint8_t *PORTA = ( uint8_t * ) 0x0002; /* Non-compliant */
uint16_t *p;
int32_t addr = ( int32_t ) &p; /* Non-compliant */
uint8_t *q = ( uint8_t * ) addr; /* Non-compliant */
bool_t b = ( bool_t ) p; /* Non-compliant */
enum etag { A, B } e = ( enum etag ) p; /* Non-compliant */

```

Rule 11.5

Không nên thực hiện chuyển đổi từ con trỏ void sang con trỏ đối tượng

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chuyển đổi một con trỏ void sang con trỏ đối tượng có thể tạo ra con trỏ không được căn chỉnh đúng, dẫn tới hành vi không xác định.

```

uint32_t *p32;
void *p;
uint16_t *p16;
p = p32; /* Compliant - pointer to uint32_t into
* pointer to void */
p16 = p; /* Non-compliant */
p = ( void * ) p16; /* Compliant */
p32 = ( uint32_t * ) p; /* Non-compliant */

```

Rule 11.6

Không được thực hiện ép kiểu giữa con trỏ void và một kiểu số học (arithmetic type)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chuyển đổi một số nguyên thành con trỏ void có thể tạo ra con trỏ không được căn chỉnh đúng, dẫn tới hành vi không xác định.

```
void * p;
uint32_t u;
/* Non-compliant - implementation-defined */
p = ( void * ) 0x1234u;
/* Non-compliant - undefined */
p = ( void * ) 1024.0f;
/* Non-compliant - implementation-defined */
u = ( uint32_t ) p;
```

Rule 11.7

Không được thực hiện ép kiểu giữa con trỏ đối tượng và một kiểu số học không nguyên

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chuyển đổi kiểu cơ bản là Boolean, character, hoặc enum thành con trỏ đối tượng có thể tạo ra con trỏ không được căn chỉnh đúng, dẫn tới hành vi không xác định.

```
int16_t * p;
float32_t f;
f = ( float32_t ) p; /* Non-compliant */
p = ( int16_t * ) f; /* Non-compliant */
```

Rule 11.8

Một phép ép kiểu không được loại bỏ bất kỳ định tính const hoặc volatile nào khỏi kiểu mà một con trỏ trỏ tới

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Bất kỳ nỗ lực nào nhằm loại bỏ định tính (qualification) liên kết với kiểu được trỏ tới bằng cách ép kiểu đều vi phạm nguyên tắc định tính kiểu.

```
uint16_t x;
uint16_t * const cpi = &x; // const pointer to uint16_t
uint16_t * const *pcpi; // pointer to const pointer to uint16_t
uint16_t **ppi; // pointer to pointer to uint16_t
const uint16_t *pci; // pointer to const uint16_t
volatile uint16_t *pvi; // pointer to volatile uint16_t
uint16_t *pi; // pointer to uint16_t

pi = cpi; // Compliant - no conversion, no cast required
pi = (uint16_t *)pci; // Non-compliant
pi = (uint16_t *)pvi; // Non-compliant
ppi = (uint16_t ***)pcpi; // Non-compliant
```

Rule 11.9

Macro NULL là hình thức duy nhất được phép sử dụng cho hằng con trỏ null kiểu số nguyên

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Sử dụng NULL thay vì 0 giúp thể hiện rõ ràng rằng một hằng con trỏ null được chủ đích sử dụng.

Sử dụng trực tiếp 0, như trong MY_NULL_1, là không tuân thủ vì nó không thể hiện rõ ràng một hằng con trỏ null.

```
int32_t *p1 = 0; /* Non-compliant */
int32_t *p2 = ( void * ) 0; /* Compliant */

#define MY_NULL_1 0
#define MY_NULL_2 ( void * ) 0
if ( p1 == MY_NULL_1 ) /* Non-compliant */
{
}
if ( p2 == MY_NULL_2 ) /* Compliant */
{
}
```

Cách dùng đúng: MY_NULL_2, mở rộng thành (void *)0, là tuân thủ vì nó biểu diễn tường minh một hằng con trỏ null.

Rule 12.1

Thứ tự ưu tiên của các toán tử trong biểu thức nên được thể hiện tường minh

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nên dùng dấu ngoặc để thể hiện rõ thứ tự ưu tiên mong muốn giữa các toán tử, thay vì chỉ dựa vào quy tắc ưu tiên mặc định của ngôn ngữ C, giúp mã dễ đọc và tránh nhầm lẫn.

Rule 12.2

Toán hạng bên phải của một toán tử dịch bit phải nằm trong khoảng từ 0 đến nhỏ hơn một đơn vị so với độ rộng bit của kiểu cơ bản của toán hạng bên trái

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu toán hạng bên phải là số âm, hoặc lớn hơn hay bằng độ rộng của toán hạng bên trái, hành vi là không xác định.

```
u8a = u8a << 7; /* Compliant */
u8a = u8a << 8; /* Non-compliant */
u16a = ( uint16_t ) u8a << 9; /* Compliant */
```

Cần lưu ý rằng kiểu cơ bản của 1u là unsigned char, trong khi kiểu cơ bản của 1UL là unsigned long:

```
1u << 10u; /* Non-compliant */
( uint16_t ) 1u << 10u; /* Compliant */
1UL << 10u; /* Compliant */
```

Rule 12.3

Không nên sử dụng toán tử dấu phẩy (comma operator)

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Việc sử dụng toán tử dấu phẩy thường gây bất lợi cho khả năng đọc mã, và hiệu quả tương tự thường có thể đạt được bằng cách khác.

```
f ( ( 1, 2 ), 3 ); /* Non-compliant - how many parameters? */
```

Rule 12.4

Việc tính giá trị của biểu thức hằng không nên dẫn tới tràn số vòng (wrap-around) đối với số nguyên không dấu

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Biểu thức số nguyên không dấu không tràn (overflow) theo nghĩa chặt chẽ, mà thay vào đó wrap-around (vòng lại). Mặc dù có lý do chính đáng để dùng số học modulo lúc runtime, việc này ít khi là chủ đích tại thời điểm biên dịch.

```
#define DELAY 10000u
#define WIDTH 60000u
void fixed_pulse ( void )
{
    uint16_t off_time16 = DELAY + WIDTH; /* Non-compliant */
}
```

Trong ví dụ này, biểu thức `DELAY + WIDTH` có giá trị 70000 nhưng sẽ wrap-around thành 4464 trên máy có kiểu `int` 16-bit (65535 là giá trị lớn nhất của số nguyên không dấu 16-bit).

Rule 13.1

Danh sách khởi tạo (initializer list) không được chứa side effect có tính bền vững (persistent side effects)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C99

C90 giới hạn bộ khởi tạo cho các đối tượng tự động có kiểu aggregate chỉ được chứa biểu thức hằng. Tuy nhiên, C99 cho phép bộ khởi tạo aggregate tự động chứa các biểu thức được tính lúc runtime.

```
volatile uint16_t v1;
void f ( void )
{
    /* Non-compliant - volatile access is a persistent side effect */
    uint16_t a[ 2 ] = { v1, 0 };
}
void g ( uint16_t x, uint16_t y )
{
    /* Compliant - no side effects */
    uint16_t a[ 2 ] = { x + y, x - y };
}
```

Rule 13.2

Giá trị của một biểu thức và các side effect bền vững của nó phải giống nhau ở mọi thứ tự tính toán được cho phép

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

1. Không đối tượng nào được chỉnh sửa nhiều hơn một lần;
2. Không đối tượng nào vừa được chỉnh sửa vừa được đọc, trừ khi việc đọc đó góp phần tính toán giá trị được lưu vào chính đối tượng;
3. Không có quá một lần truy cập chỉnh sửa với kiểu `volatile-qualified`;
4. Không có quá một lần truy cập đọc với kiểu `volatile-qualified`.

```
int i = i++ + 1; // i is modified twice within the same expression

i = i + (i = 2); // i is read and modified in an unpredictable manner
volatile int v;
```

```
v = v + 1; // This might involve multiple reads and writes, leading to issues
volatile int v;
int a = v + v; // Two read accesses to the volatile variable 'v'
```

Rule 13.3

Một biểu thức đầy đủ chứa toán tử tăng (++) hoặc giảm (--) không nên có side effect nào khác ngoài side effect do chính toán tử tăng/giảm gây ra

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Việc dùng toán tử tăng/giảm kết hợp với các toán tử khác không được khuyến nghị vì:

- Có thể làm giảm đáng kể khả năng đọc của mã;
- Đưa thêm side effect vào câu lệnh, tiềm ẩn nguy cơ hành vi không xác định (theo Quy tắc 13.2).

Sẽ rõ ràng hơn nếu tách các phép toán này khỏi các toán tử khác.

```
/* Biểu thức: u8a = u8b++; là không tuân thủ.
Câu lệnh biểu thức: u8a = ++u8b + u8c--; không tuân thủ.
Nên viết rõ ràng hơn theo trình tự sau: */
++u8b;
u8a = u8b + u8c;
u8c--;
```

Rule 13.4

Không nên sử dụng kết quả của toán tử gán (assignment operator)

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Việc sử dụng toán tử gán, đơn giản hay kết hợp, cùng với các toán tử số học khác không được khuyến nghị vì:

- Có thể làm giảm đáng kể khả năng đọc của mã;
- Đưa thêm side effect vào câu lệnh, khiến việc tránh hành vi không xác định (theo Quy tắc 13.2) khó khăn hơn.

```
x = y; /* Compliant */
a[ x ] = a[ x = y ]; /* Non-compliant - the value of x = y is used */
if ( bool_var = false )
{
}
/* Non-compliant - value of bool_var = false is used but
* bool_var == false was probably intended */
```

Thực hành tốt để duy trì sự rõ ràng và tránh lỗi tiềm ẩn: dùng = để gán (x = y;), dùng == để so sánh (if (bool_var == false)).

Rule 13.5

Toán hạng bên phải của toán tử logic && hoặc || không được chứa side effect bền vững

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Nếu việc tính toán hạng bên phải tạo ra side effect không bền vững tại điểm biểu thức xuất hiện trong chương trình, thì việc toán hạng bên phải có được tính hay không sẽ không quan trọng.

```
void g ( void )
{
```

```

/* Compliant - f ( ) has no persistent side effects */
if ( is_high && ( a == f ( x ) ) )
{
}
/* Non-compliant - h ( ) has a persistent side effect */
if ( is_high && ( a == h ( x ) ) )
{
}
}

```

Biểu thức logic `is_high && (a == h(x))` có vấn đề vì `h(x)` làm thay đổi trạng thái (state) trong quá trình thực thi.

Rule 13.6

Toán hạng của toán tử `sizeof` không được chứa bất kỳ biểu thức nào có side effect tiềm ẩn

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Toán hạng của toán tử `sizeof` có thể là một biểu thức hoặc chỉ định một kiểu. Nếu toán hạng chứa biểu thức, một lỗi lập trình có thể xảy ra là kỳ vọng biểu thức đó được tính giá trị, trong khi thực tế phần lớn trường hợp nó không được tính.

```

volatile int32_t i;
int32_t j;
size_t s;
s = sizeof ( j ); /* Compliant */
s = sizeof ( j++ ); /* Non-compliant */
s = sizeof ( i ); /* Compliant - exception */
s = sizeof ( int32_t ); /* Compliant */

```

Rule 14.1

Biến đếm vòng lặp không được có kiểu cơ bản là floating

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Khi sử dụng biến đếm vòng lặp kiểu dấu phẩy động, sai số làm tròn tích lũy có thể dẫn tới sự chênh lệch giữa số lần lặp mong đợi và thực tế. Điều này có thể xảy ra khi bước lặp không phải lũy thừa của cơ số dấu phẩy động bị làm tròn về một giá trị biểu diễn được.

```

uint32_t counter = 0u;
for ( float32_t f = 0.0f; f < 1.0f; f += 0.001f )
{
  ++counter;
}

```

Rule 14.2

Vòng lặp for phải được viết đúng cấu trúc (well-formed)

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Câu lệnh for cung cấp cơ chế lặp đa dụng. Sử dụng một dạng vòng lặp giới hạn giúp mã dễ review và phân tích hơn.

```

#include <stdio.h>

int main() {
  // Well-formed for loop
  for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
  }
}

```

```
}  
    return 0;  
}
```

Trong vòng lặp well-formed này: Khởi tạo — biến điều khiển vòng lặp *i* được khởi tạo về 0. Điều kiện — vòng lặp tiếp tục chạy khi *i* nhỏ hơn 10. Bước lặp — biến điều khiển *i* tăng thêm 1 sau mỗi lần lặp.

Rule 14.3

Biểu thức điều khiển (controlling expressions) không được có giá trị bất biến (invariant)

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Nếu một biểu thức điều khiển có giá trị bất biến, có khả năng đây là lỗi lập trình. Mã không thể truy cập được do sự hiện diện của biểu thức bất biến có thể bị trình biên dịch loại bỏ, dẫn đến việc loại bỏ mã phòng vệ (defensive code) khỏi chương trình thực thi.

```
if ( 2 > 3 )  
{  
    /* Non-compliant - always false */  
}  
for ( s8a = 0; s8a < 130; ++s8a )  
{  
    /* Non-compliant - always true */  
}  
if ( ( s8a < 10 ) && ( s8a > 20 ) )  
{  
    /* Non-compliant - always false */  
}
```

Rule 14.4

Biểu thức điều khiển của câu lệnh if và biểu thức điều khiển của câu lệnh lặp phải có kiểu cơ bản là Boolean

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Kiểu hóa chặt chẽ (strong typing) đòi hỏi biểu thức điều khiển của câu lệnh if hoặc câu lệnh lặp phải có kiểu cơ bản là Boolean.

```
int32_t *p, *q;  
while ( p ) /* Non-compliant - p is a pointer */  
{  
}  
while ( q != NULL ) /* Compliant */  
{  
}  
while ( true ) /* Compliant */  
{  
}
```

Rule 15.1

Không nên sử dụng câu lệnh goto

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Sử dụng goto không giới hạn có thể dẫn tới chương trình thiếu cấu trúc và cực kỳ khó hiểu. Xem thêm hướng dẫn tại Quy tắc 15.2 và 15.3.

Rule 15.2

Câu lệnh goto chỉ được nhảy tới một nhãn được khai báo sau đó, trong cùng một hàm

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Sử dụng goto không giới hạn có thể dẫn tới chương trình thiếu cấu trúc và cực kỳ khó hiểu.

```
void f ( void )
{
    int32_t j = 0;
L1:
    ++j;
    if ( 10 == j )
    {
        goto L2; /* Compliant */
    }
    goto L1; /* Non-compliant */
L2 :
    ++j;
}
```

Rule 15.3

Bất kỳ nhãn nào được tham chiếu bởi câu lệnh goto phải được khai báo trong cùng khối, hoặc trong bất kỳ khối nào bao quanh câu lệnh goto

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Sử dụng goto không giới hạn có thể dẫn tới chương trình thiếu cấu trúc và cực kỳ khó hiểu.

Việc ngăn nhảy giữa các khối, hoặc vào các khối lồng nhau, giúp giảm thiểu độ phức tạp trực quan của mã.

```
switch ( x )
{
    case 0:
        if ( x == y )
        {
            goto L1;
        }
        break;
    case 1:
        y = x;
L1:
        ++x;
        break;
    default:
        break;
}
```

Trong ví dụ sau, nhãn L1 được định nghĩa trong một khối bao quanh khối chứa câu lệnh goto. Tuy nhiên, việc nhảy vượt từ switch-clause này sang switch-clause khác, và do switch-clause được coi như các khối theo quy tắc này, câu lệnh goto này là không tuân thủ.

Rule 15.4

Không nên có nhiều hơn một câu lệnh break hoặc goto được dùng để kết thúc một câu lệnh lặp

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Hạn chế số lượng điểm thoát khỏi vòng lặp giúp giảm thiểu độ phức tạp trực quan của mã. Việc dùng một câu lệnh break hoặc goto cho phép tạo một đường thoát phụ duy nhất khi cần kết thúc vòng lặp sớm.

```
for ( x = 0; x < LIMIT; ++x )
{
    if ( ExitNow ( x ) )
    {
        break;
    }
    for ( y = 0; y < x; ++y )
    {
        if ( ExitNow ( LIMIT - y ) )
        {
            break;
        }
    }
}
```

Cả hai vòng lặp lồng nhau trên đều tuân thủ vì mỗi vòng chỉ dùng một break duy nhất để kết thúc sớm.

Rule 15.5

Một hàm nên có một điểm thoát duy nhất ở cuối hàm

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu một hàm có các điểm thoát nằm xen kẽ với các câu lệnh tạo ra side effect bền vững, sẽ không dễ để xác định side effect nào sẽ xảy ra khi hàm được thực thi.

```
bool_t f ( uint16_t n, char *p )
{
    if ( n > MAX )
    {
        return false;
    }
    if ( p == NULL )
    {
        return false;
    }
    return true;
}
```

Trong ví dụ không tuân thủ trên, các return sớm được dùng để kiểm tra tham số của hàm.

Rule 15.6

Phần thân của một câu lệnh lặp hoặc câu lệnh lựa chọn phải là một câu lệnh hợp thành (compound-statement)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Phần thân của câu lệnh lặp (while, do...while hoặc for) hoặc câu lệnh lựa chọn (if, else, switch) phải là một câu lệnh hợp thành (được bao trong { }).

```
/* Hành vi thực tế: */
if ( flag_1 )
{
    if ( flag_2 )
    {
        action_1 ( );
    }
    else
```

```
{
action_2 ( );
}
}
/* Trong khi cách viết dưới đây dễ gây nhầm lẫn: */
if ( flag_1 )
if ( flag_2 ) /* Non-compliant */
action_1 ( ); /* Non-compliant */
else
action_2 ( ); /* Non-compliant */
```

Rule 15.7

Mọi cấu trúc if ... else if phải được kết thúc bằng một câu lệnh else

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Một câu lệnh else cuối cùng phải luôn được cung cấp bất cứ khi nào một câu lệnh if được theo sau bởi một hoặc nhiều cấu trúc else if. Câu lệnh else đó phải chứa ít nhất một side effect hoặc một chú thích.

```
if ( flag_1 )
{
action_1 ( );
}
else if ( flag_2 )
{
action_2 ( );
}
/* Non-compliant */
/* Cách viết tuân thủ với else kết thúc: */
else
{
; /* No action required - ; is optional */
}
```

Rule 16.1

Mọi câu lệnh switch phải được viết đúng cấu trúc (well-formed)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Cú pháp của câu lệnh switch trong C không thực sự chặt chẽ và có thể cho phép hành vi phức tạp, thiếu cấu trúc. Quy tắc này cùng các quy tắc khác áp đặt một cấu trúc đơn giản và nhất quán lên câu lệnh switch.

Các quy tắc còn lại trong mục này đưa ra các ví dụ cũng liên quan tới quy tắc này.

Rule 16.2

Một nhãn switch chỉ được sử dụng khi câu lệnh hợp thành bao quanh gần nhất chính là phần thân của câu lệnh switch

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chuẩn cho phép một nhãn switch (nhãn case hoặc default) được đặt trước bất kỳ câu lệnh nào bên trong phần thân của một câu lệnh switch, có khả năng dẫn tới mã thiếu cấu trúc. Để ngăn điều này, một nhãn switch chỉ nên xuất hiện ở cấp ngoài cùng của câu lệnh hợp thành tạo nên phần thân câu lệnh switch.

```
switch ( x )
{
case 1: /* Compliant */
if ( flag )
```

```
{
case 2: /* Non-compliant */
x = 1;
}
break;
default:
break;
}
```

Rule 16.3

Một câu lệnh break vô điều kiện phải kết thúc mỗi switch-clause

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu switch-clause là một câu lệnh hợp thành, câu lệnh cuối cùng trong câu lệnh hợp thành đó phải là một câu lệnh break.

```
switch ( x )
{
case 0:
break; /* Compliant - unconditional break */
case 1: /* Compliant - empty fall through allows a group */
case 2:
break; /* Compliant */
case 4:
a = b; /* Non-compliant - break omitted */
case 5:
if ( a == b )
{
++a;
break; /* Non-compliant - conditional break */
}
break; /* Compliant - moved unconditional break outside of if */
default:
; /* Non-compliant - default must also have a break */
}
```

Rule 16.4

Mọi câu lệnh switch phải có một nhãn default

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Yêu cầu có nhãn default là một hình thức lập trình phòng vệ. Các câu lệnh theo sau nhãn default được chủ đích thực hiện một hành động phù hợp.

```
int16_t x;
switch ( x )
{
case 0:
++x;
break;
case 1:
case 2:
break;
/* Non-compliant - default label is required */
}
default:
break; /* Compliant - added unconditional break */
```

Rule 16.5

Nhãn default phải xuất hiện là nhãn switch đầu tiên hoặc cuối cùng của câu lệnh switch

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Quy tắc này giúp dễ dàng xác định vị trí nhãn default trong một câu lệnh switch.

```
switch ( x )
{
default: /* Compliant - default is the first label */
case 0:
++x;
break;
case 1:
case 2:
break;
}
```

Rule 16.6

Mọi câu lệnh switch phải có ít nhất hai switch-clause

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Một câu lệnh switch chỉ có một nhánh là dư thừa và có thể là dấu hiệu của lỗi lập trình.

```
switch ( z )
{
case 1:
z = 2;
break;
default: /* Compliant */
z = 0;
break;
}
switch ( x )
{
default: /* Non-compliant - switch is redundant */
x = 0;
break;
}
```

Rule 16.7

Biểu thức switch không được có kiểu cơ bản là Boolean

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chuẩn yêu cầu biểu thức điều khiển của câu lệnh switch phải có kiểu số nguyên. Vì kiểu dùng để triển khai giá trị Boolean là số nguyên, nên có thể có câu lệnh switch được điều khiển bởi biểu thức Boolean. Trong trường hợp này, cấu trúc if-else sẽ phù hợp hơn.

```
switch ( x == 0 ) /* Non-compliant - essentially Boolean */
{ /* In this case an "if-else" would be more logical */
case false:
y = x;
break;
default:
y = z;
break;
}
```

Session 4 — Quy tắc 96 đến 125

Nội dung phiên 4

- Hàm (Function)
- Con trỏ và mảng (Pointer and array)
- Quản lý tài nguyên (Resource management)
- Chỉ thị tiền xử lý (Preprocessing directives)

Rule 17.1

Tiện ích `stdarg.h` không được sử dụng

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Tiện ích `stdarg.h` cho phép định nghĩa các hàm nhận số lượng tham số biến đổi (variadic functions). Việc sử dụng cơ chế này rất dễ xảy ra lỗi vì trình biên dịch không thể kiểm tra kiểu và số lượng của các tham số truyền vào.

```
#include <stdarg.h>
#include <stdio.h>

void print_numbers(int count, ...) {
    va_list args;
    va_start(args, count);
    for (int i = 0; i < count; i++) {
        int num = va_arg(args, int);
        printf("%d\n", num);
    }
    va_end(args);
}
```

Rule 17.2

Một hàm không được gọi lại chính nó, dù trực tiếp hay gián tiếp

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Đệ quy (recursion), dù trực tiếp hay gián tiếp, không được khuyến khích vì có thể dẫn tới việc sử dụng bộ nhớ stack không giới hạn, sự cạn kiệt tài nguyên, và khiến việc phân tích luồng dữ liệu và bộ nhớ trở nên khó khăn hơn.

```
/* Đệ quy trực tiếp - không tuân thủ */
uint16_t factorial ( uint16_t x )
{
    if ( x == 0u )
    {
        return 1u;
    }
    return x * factorial ( x - 1u ); /* Non-compliant */
}
/* Đệ quy gián tiếp - không tuân thủ */
void func1 ( void )
{
    func2 ( ); /* Non-compliant */
}
void func2 ( void )
{
    func1 ( ); /* Non-compliant */
}
```

Rule 17.3

Một hàm không được gọi mà không có prototype nhìn thấy được (visible prototype)

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90

Nếu gọi một hàm mà không có prototype nhìn thấy được tại điểm gọi, trình biên dịch không thể kiểm tra rằng lời gọi có phù hợp với định nghĩa hàm hay không.

```
extern void func1 ( int16_t p1 ); /* Prototype declaration */
void func2 ( void )
{
    func1 ( 12 ); /* Compliant - prototype visible */
    func3 ( 1, 2 ); /* Non-compliant - no visible prototype */
}
```

Rule 17.4

Tất cả các đường dẫn thoát khỏi một hàm khác kiểu void phải có một câu lệnh return với biểu thức tường minh

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Nếu hàm không có kiểu trả về void, việc sử dụng giá trị trả về (không xác định) từ hàm bất cứ khi nào một đường dẫn thoát không có return biểu thức tường minh đều là hành vi không xác định.

```
int16_t Foo ( uint16_t x )
{
    if ( x == 1u )
    {
        return 1; /* Compliant */
    }
    else
    {
        /* Non-compliant - missing return statement */
    }
}
```

Rule 17.5

Nếu tham số hàm là một mảng, số lượng phần tử của đối số phải khớp với số lượng phần tử của kiểu tham số hàm

Phân loại: Khuyến nghị (Advisory) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Khi một tham số hàm được khai báo như một mảng có kích thước, chuẩn quy định rằng khai báo tham số đó được chuyển đổi thành một con trỏ. Việc dùng cú pháp mảng khi khai báo tham số hàm là một cách hữu ích để thể hiện ý định của lập trình viên về số phần tử mà đối số cần có.

```
void fn1 ( int16_t arr[ 3 ] ) /* Compliant */
{
}
void fn2 ( void )
{
    int16_t a1[ 3 ];
    int16_t a2[ 4 ];
    fn1 ( a1 ); /* Compliant - matches size of a1 */
    fn1 ( a2 ); /* Non-compliant - a2 has 4 elements */
}
```

Rule 17.6

Từ khóa *static* không được sử dụng trong khai báo bộ định lượng mảng (array index) của một tham số hàm

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C99

`static` trong khai báo chỉ định mảng của tham số hàm được dùng để chỉ ra rằng con trỏ đối số phải trở tới ít nhất số phần tử đã chỉ định. Không phải mọi trình biên dịch đều hỗ trợ hoặc thực thi ràng buộc này một cách nhất quán, và cú pháp này thường bị hiểu nhầm.

```
void fn ( int32_t arr[ static 3 ] ) /* Non-compliant */
{
}
```

Rule 17.7

Giá trị trả về của một hàm có kiểu khác *void* phải được sử dụng

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu một hàm được thiết kế để trả về một giá trị, thông thường giá trị đó cần phải được kiểm tra hoặc sử dụng — ví dụ mã trạng thái lỗi trả về từ một hàm thư viện.

```
extern int16_t func1 ( void );
void func2 ( void )
{
    func1 ( ); /* Non-compliant - return value discarded */
    ( void ) func1 ( ); /* Compliant - by exception, explicitly discarded */
}
```

Rule 17.8

Một tham số hàm không nên được chỉnh sửa

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chỉnh sửa giá trị của một tham số hàm (được truyền theo giá trị) không ảnh hưởng tới đối số truyền vào từ hàm gọi, nhưng có thể gây nhầm lẫn cho người đọc, nhất là nếu tham số được dùng để lưu trữ một giá trị khác trong quá trình thực thi hàm.

```
void f ( uint16_t x )
{
    x = x + 1u; /* Non-compliant */
}
```

Rule 18.1

Một phép tính số học con trỏ chỉ được áp dụng cho con trỏ trỏ tới một phần tử của một đối tượng mảng

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Cộng hoặc trừ một con trỏ và một số nguyên chỉ hợp lệ nếu con trỏ trỏ tới một phần tử của mảng (hoặc tới vị trí ngay sau phần tử cuối cùng của mảng).

```
int16_t array[ 10 ];
int16_t *p1 = array;
int16_t *p2 = p1 + 5; /* Compliant - within array bounds */
int16_t *p3 = p1 + 15; /* Non-compliant - outside array bounds */
```

Rule 18.2

Phép trừ không nên được áp dụng cho hai con trỏ không trỏ tới phần tử của cùng một đối tượng mảng

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Trừ hai con trỏ chỉ có ý nghĩa nếu cả hai đều trỏ tới phần tử của cùng một mảng (hoặc vị trí ngay sau phần tử cuối cùng của cùng mảng đó).

```
int16_t arr1[ 10 ];
int16_t arr2[ 10 ];
int16_t *p1 = &arr1[ 5 ];
int16_t *p2 = &arr2[ 3 ];
ptrdiff_t diff = p1 - p2; /* Non-compliant - different arrays */
```

Rule 18.3

Các toán tử quan hệ (<, <=, >, >=) không nên được áp dụng cho con trỏ trỏ tới các đối tượng khác nhau

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

So sánh quan hệ giữa các con trỏ không trỏ tới phần tử của cùng một mảng dẫn tới hành vi không xác định.

```
int16_t a;
int16_t b;
int16_t *p1 = &a;
int16_t *p2 = &b;
if ( p1 < p2 ) /* Non-compliant - a and b are different objects */
{
}
```

Rule 18.4

Toán tử +, -, += và -= không nên được áp dụng cho một biểu thức có kiểu con trỏ

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Số học con trỏ có thể khó đọc và dễ xảy ra lỗi. Nên ưu tiên sử dụng cú pháp chỉ số mảng (array indexing) rõ ràng hơn thay vì thao tác trực tiếp trên con trỏ.

```
int16_t arr[ 10 ];
int16_t *p = arr;
p = p + 2; /* Non-compliant - pointer arithmetic used */
int16_t x = arr[ 2 ]; /* Compliant - array indexing used instead */
```

Rule 18.5

Các khai báo nên chứa không quá 2 cấp lồng nhau của con trỏ dẫn xuất (derived pointer)

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Các cấp lồng nhau (nesting) của con trỏ vượt quá 2 cấp làm cho mã khó hiểu và bảo trì hơn nhiều.

```
int16_t **p1; /* Compliant - 2 levels */
int16_t ***p2; /* Non-compliant - 3 levels */
```

Rule 18.6

Con trỏ trỏ tới một đối tượng có thời gian sống lưu trữ tự động không được sử dụng sau khi đối tượng đó hết vòng đời

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Sau khi một đối tượng có thời gian sống lưu trữ tự động (automatic storage duration) kết thúc vòng đời, bất kỳ con trỏ nào vẫn còn trỏ tới đối tượng đó đều trở thành con trỏ không hợp lệ (dangling pointer).

```
int16_t *dangling_pointer ( void )
{
    int16_t local = 10;
    return &local; /* Non-compliant - local goes out of scope */
}
```

Rule 18.7

Không nên khai báo kiểu mảng linh hoạt (flexible array member)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C99

Mảng linh hoạt (flexible array member) là một cấu trúc chuyên biệt, khó phân tích tĩnh và có thể dẫn tới lỗi truy cập bộ nhớ nếu bị sử dụng sai.

```
struct s {
    int16_t n;
    int16_t data[ ]; /* Non-compliant - flexible array member */
};
```

Rule 18.8

Không được khai báo kiểu mảng có độ dài biến đổi (variable-length array)

Phân loại: Bắt buộc tuyệt đối (Mandatory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C99

Mảng có độ dài biến đổi (VLA) được cấp phát trên stack với kích thước xác định lúc runtime, điều này có thể dẫn tới tràn stack nếu kích thước không được kiểm soát chặt chẽ.

```
void f ( uint16_t n )
{
    int16_t vla[ n ]; /* Non-compliant - variable-length array */
}
```

Rule 19.1

Một đối tượng không được gán hoặc sao chép tới một đối tượng chồng lấp (overlapping object) khác

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Hệ thống | Áp dụng: C90, C99

Nếu vùng nhớ của đối tượng nguồn và đích chồng lấp nhau, hành vi của phép gán hoặc sao chép (ví dụ memcpy) là không xác định.

```
char buf[ 10 ] = "abcdefghi";
memcpy ( &buf[ 0 ], &buf[ 2 ], 5 ); /* Non-compliant - overlapping regions */
memmove ( &buf[ 0 ], &buf[ 2 ], 5 ); /* Compliant - memmove handles overlap safely */
```

Rule 19.2

Từ khóa union không nên được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Việc sử dụng union cho phép cùng một vùng nhớ được diễn giải theo nhiều kiểu khác nhau, điều này có thể dẫn tới hành vi phụ thuộc cách triển khai hoặc không xác định nếu không được sử dụng cẩn thận.

```
union u_tag {
    int16_t i;
    float32_t f;
}; /* Non-compliant */
```

Rule 20.1

Các chỉ thị #include chỉ nên được đặt trước các phần tử khai báo (declaration) hoặc chỉ thị tiền xử lý khác

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Đặt các chỉ thị #include ở đầu tệp giúp dễ dàng xác định các phụ thuộc của tệp và tránh các vấn đề tiềm ẩn liên quan tới thứ tự include.

```
#include <stdio.h> /* Compliant - at top of file */
int16_t x;
#include <stdlib.h> /* Non-compliant - after a declaration */
```

Rule 20.2

Các ký tự '/*', '//', '\\ hoặc " không nên xuất hiện trong tên tệp header (#include filename)

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Ý nghĩa của các ký tự này bên trong tên tệp trong chỉ thị #include là phụ thuộc cách triển khai (implementation-defined), và có thể gây ra lỗi biên dịch không mong muốn hoặc hành vi không nhất quán giữa các trình biên dịch.

```
#include "sub\dir\header.h" /* Non-compliant - contains backslash */
#include "sub/dir/header.h" /* Compliant */
```

Rule 20.3

Cú pháp chỉ thị #include phải được tuân thủ đúng

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Chỉ thị #include phải có dạng #include <filename> hoặc #include "filename", đúng theo cú pháp chuẩn quy định.

```
#define HEADER_NAME "myheader.h"
#include HEADER_NAME /* Non-compliant in some interpretations - use with caution */
#include "myheader.h" /* Compliant */
```

Rule 20.4

Một macro không nên được định nghĩa lại với cùng tên như một từ khóa

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Định nghĩa lại một từ khóa bằng macro có thể khiến mã trở nên rất khó hiểu và dẫn tới hành vi không mong muốn.

```
#define int short /* Non-compliant - redefining a keyword */
```

Rule 20.5

Chỉ thị `#undef` không nên được sử dụng

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Việc undefine rồi định nghĩa lại một macro có thể gây khó khăn cho việc theo dõi giá trị thực tế của macro tại từng điểm trong mã nguồn.

```
#define MAX_SIZE 100
#undef MAX_SIZE /* Non-compliant */
#define MAX_SIZE 200
```

Rule 20.6

Token `#` hoặc `##` không nên xuất hiện trong đối số của macro

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Sử dụng các token này bên trong đối số của macro có thể tạo ra hành vi không xác định hoặc không nhất quán khi macro được mở rộng.

```
#define CONCAT(a, b) a##b
CONCAT(x, #y) /* Non-compliant - '#' used within a macro argument */
```

Rule 20.7

Các tham số macro dạng biểu thức phải được đặt trong dấu ngoặc đơn

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu tham số của macro không được đặt trong ngoặc đơn khi sử dụng trong phần thân macro, kết quả mở rộng macro có thể cho ra một biểu thức khác với ý định ban đầu do thứ tự ưu tiên toán tử.

```
#define SQUARE(x) x * x /* Non-compliant */
SQUARE(1 + 2) /* Expands to 1 + 2 * 1 + 2, not 9 */

#define SQUARE(x) ((x) * (x)) /* Compliant */
SQUARE(1 + 2) /* Expands to ((1 + 2) * (1 + 2)) = 9 */
```

Rule 20.8

Biểu thức điều khiển của chỉ thị `#if` hoặc `#elif` phải có kiểu bool về bản chất

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Biểu thức điều khiển của `#if` hoặc `#elif` nên được viết sao cho kết quả về bản chất mang tính Boolean, giúp mã rõ ràng và dễ đọc hơn.

```
#if MAX_SIZE /* Non-compliant - not essentially Boolean */
#endif
#if MAX_SIZE > 0 /* Compliant */
#endif
```

Rule 20.9

Tất cả các định danh được sử dụng trong biểu thức điều khiển của `#if` hoặc `#elif` phải được định nghĩa trước khi được đánh giá

Phân loại: Bắt buộc (Required) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu một macro chưa được định nghĩa được sử dụng trong biểu thức `#if`, nó sẽ được thay thế bằng giá trị 0, điều này có thể dẫn tới hành vi không như mong đợi.

```
#if UNDEFINED_MACRO /* Non-compliant - undefined identifier */
#endif
```

Rule 20.10

Không nên sử dụng toán tử `#` và `##`

Phân loại: Khuyến nghị (Advisory) | Phân tích: Có thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Toán tử stringizing (`#`) và token-pasting (`##`) có thể tạo ra mã khó đọc và khó gỡ lỗi, do cách mở rộng macro không trực quan.

```
#define STRINGIZE(x) #x /* Non-compliant */
#define CONCAT(a, b) a##b /* Non-compliant */
```

Rule 20.11

Nếu toán tử `##` được áp dụng cho một đối số macro thuộc dạng chỉ định thay thế cho toán tử `#` thì kết quả nối token phải là một token hợp lệ

Phân loại: Bắt buộc (Required) | Phân tích: Không thể quyết định, Đơn vị biên dịch đơn | Áp dụng: C90, C99

Nếu kết quả của việc nối hai token bằng `##` không tạo thành một token hợp lệ theo chuẩn C, hành vi là không xác định.

```
#define CONCAT(a, b) a##b
CONCAT(1, +) /* Non-compliant - "1+" is not a valid token */
CONCAT(x, y) /* Compliant - "xy" is a valid identifier token */
```